Linear Algebra

Broadcasting

Add vector $\boldsymbol{b}$ to every column of matrix A:

$$C = A + \boldsymbol{b}, \quad C_{\{i,j\}} = A_{\{i,j\}} + b_j$$

Matrix multiplication

For vectors:

$$x^T y = \|x\|_2 \ \|y\|_2 \cos(\theta)$$

Hadamard product: Element-wise multiplication: $\boldsymbol{A} \odot \boldsymbol{B}$

Norms

- $\boldsymbol{L^p}$ **norm**: $\|\boldsymbol{x}\|_p = \left(\sum_i |x_i|^p\right)^{\frac{1}{p}}$
- $\boldsymbol{L^\infty}$ **norm**: maximum element of vector, $\|\boldsymbol{x}\|_\infty = \max_i |x_i|$
- **Frobenius norm**: L^2 norm for matrices, $\|A\|_F = \sqrt{\sum_{i,j} A_{i,j}^2}$

Diagonal Matrices

A Diagonal matrix is all zero except for its diagonal. The diagonal we call vector $\boldsymbol{v}$:

$$\boldsymbol{v} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

$$\text{diag}(\boldsymbol{v}) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

Multiplying a diagonal matrix with a vector is computationally efficient:

$$\text{diag}(\boldsymbol{v})\boldsymbol{x} = \boldsymbol{v} \odot \boldsymbol{x}$$

Inverting diagonal matrices

$$\text{diag}(\boldsymbol{v})^{-1} = \text{diag}\left(\begin{pmatrix} \frac{1}{v_1} & \cdots & \frac{1}{v_n} \end{pmatrix}^T\right)$$

Orthogonal matrices

$$\mathbf{A}^T \ \mathbf{A} = \mathbf{A} \ \mathbf{A}^T = \boldsymbol{I}$$
$$\mathbf{A}^{-1} = \mathbf{A}^T$$

Eigenvectors

A **real symmetric matrix** with **n rows and columns** has **n real eigenvalues** and **n orthogonal eigenvectors**.

Eigenvectors can be concatenated into a n orthogonal matrix:

$$\boldsymbol{V} = \begin{pmatrix} \boldsymbol{v}^{(1)} & \cdots & \boldsymbol{v}^{(n)} \end{pmatrix}$$

The eigenvalues can be concatenated into a vector:

$$\boldsymbol{\lambda} = \begin{pmatrix} \lambda_1 & \cdots & \lambda_n \end{pmatrix}^T$$

Then the matrix $\boldsymbol{A}$ can be rewritten as:

$$\boldsymbol{A} = \boldsymbol{V}\,\mathrm{diag}(\boldsymbol{\lambda})\boldsymbol{V}^{-1}$$

Optimization using Eigendecomposition

The optimization problem of maximizing or minimizing $f(\boldsymbol{x}) = \boldsymbol{x}^T\boldsymbol{A}\boldsymbol{x}$ subject to $\|\boldsymbol{x}\|_2 = 1$ can be solved using eigendecomposition:

$$\boldsymbol{A}\boldsymbol{x} = \lambda\boldsymbol{x}$$
$$\boldsymbol{x}^T\boldsymbol{A}\boldsymbol{x} = \boldsymbol{x}^T\lambda\boldsymbol{x} = \lambda\boldsymbol{x}^T\boldsymbol{x} = \lambda$$

- **Maximum**: eigenvector belonging to the **maximum eigenvalue**.
- **Minimum**: eigenvector belonging to the **minimum eigenvalue**.

Probability Theory

Conditional Theory

$$P(\mathrm{y} = y \mid \mathrm{x} = x) = \frac{P(\mathrm{y} = y, \mathrm{x} = x)}{P(\mathrm{x} = x)}$$

Three chained together:

$$P(\mathrm{a}, \mathrm{b}, \mathrm{c}) = P(\mathrm{a} \mid \mathrm{b}, \mathrm{c})P(\mathrm{b}, \mathrm{c})$$
$$P(\mathrm{b}, \mathrm{c}) = P(\mathrm{b} \mid \mathrm{c})P(\mathrm{c})$$
$$P(\mathrm{a}, \mathrm{b}, \mathrm{c}) = P(\mathrm{a} \mid \mathrm{b}, \mathrm{c})P(\mathrm{b} \mid \mathrm{c})P(\mathrm{c})$$

Bayes Theory

$$P(\mathrm{x} \mid \mathrm{y}) = \frac{P(\mathrm{x})P(\mathrm{y} \mid \mathrm{x})}{P(\mathrm{y})}$$

Rule of total probability

$$P(\mathrm{y}) = \sum_x P(\mathrm{y} \mid x)P(x)$$

Expectation

$$E[\mathrm{x}] = \sum_x xP(x)$$
$$E[f(\mathrm{x})] = \sum_x f(x)P(x)$$

Linearity of Expectation:

$$E[a\mathrm{x} + b] = aE[\mathrm{x}] + b$$

Variance

The **variance** σ^2 measures the variability around the expectation. σ is the **standard deviation** and is the square root of the variance.

$$\mathrm{Var}(f(x)) = \mathbb{E}[(f(x) - \mathbb{E}[f(x)])^2]$$

Covariance

The covariance σ_{xy} is the linear relation between two random variables x and y.

Numerical Computation

General Methodology in Deep Learning

- m training data points $x^{(1)}, ..., x^{(m)}$
- Empirical data distribution: $\hat{p}(x) = \frac{1}{m} \sum_{i=1}^{m} \delta\left(x - x^{(i)}\right)$
- Neural Network predicts distribution: $p_{\text{model}(x;\theta)}$
  - Where θ are the learnable parameters
- Training: Make $p_{\text{model}(x)}$ as similar as possible to $\hat{p}(x)$

$\rightarrow$ Minimize **KL-divergence** (or equivalently the **cross entropy**).
$\rightarrow$ Learning is an optimization problem

Quantization

A PC only has a **finite number of bits** available, therefore we get rounding errors. The accumulation of said rounding errors can be problematic.

Underflow: The rounding of small numbers to zero. Not a direct problem, but further computation can be problematic, like $\frac{1}{x}$ or $\log(x)$ as $x \rightarrow 0$

Overflow: The rounding of large numbers to ∞. No further computation possible.

Stabilizing Softmax

For example, softmax needs to be stabilized, as very high x result in $\exp(x)$ infinity or very negative x result in $\exp(x)$ 0:

$$\text{softmax}(\boldsymbol{x})_i = \frac{\exp(x_i)}{\sum_{j=1}^{n} \exp\left(x_j\right)}$$

If we **add a constant** value **k** to all inputs, we do not change the output:

$$\frac{\exp(x_1 + k)}{\exp(x_1 + k) + \exp(x_2 + k)} = \frac{\exp(x_1)\exp(k)}{\exp(x_1)\exp(k) + \exp(x_2)\exp(k)}$$

Softmax is stabilized by subtracting the maximum input from all elements:

$$\boldsymbol{z} = \boldsymbol{x} - \max_i x_i$$

Poor Conditioning

Small change of the input has a large effect on the output.

Gradient-Based Optimization

In ML we try to minimize the cross-entropy H between the empirical distribution and the parametric model distribution. Minimizing the cross-entropy is equivalent to minimizing the KL-divergence.

$$\min_{\theta} H\left(\hat{p}_{\text{data}}, p_{\text{model}(\theta)}\right)$$

Oder anders:

$$f(x) = H\left(\hat{p}_{\text{data}}, p_{\text{model}(x)}\right)$$
$$x^* = \arg\min f(x)$$

Algorithm of Gradient Descent

1. Initialize to some point x_0
2. Calculate derivative $f'(x_0)$
3. Take small step ε in the negative direction of the derivative
   - $x_1 = x_0 - \varepsilon f'(x_0)$
   - $f(x_1) \approx f(x_0) - \varepsilon f'(x_0)$
4. Iterate until $f'(x_0) = 0$

In multidimensional space:

$$\boldsymbol{x}' = \boldsymbol{x} - \varepsilon \nabla_{\boldsymbol{x}} f(\boldsymbol{x})$$